CURSO: INTRODUCCION A PYTHON PARA CIENCIA DE DATOS
CICLO DE CURSOS: INTRODUCCIÓN A LA CIENCIA DE DATOS
CATEDRA DE MATEMÁTICA - F.C.A. – UNCUYO - 2025

---

1. Objetivos generales del curso

1. **Familiarizarse** con los fundamentos de la programación en Python, desde la instalación del entorno hasta la escritura de scripts y el uso de entornos interactivos (Jupyter Notebook, Google Colab).
2. **Desarrollar** habilidades para la manipulación, limpieza y análisis de datos con librerías fundamentales como NumPy y pandas (sin cubrir Machine Learning).
3. **Aplicar** técnicas básicas de visualización de datos con Matplotlib, enfocadas en la exploración y la comunicación de resultados.
4. **Ejecutar** un proyecto integrador que abarque la preparación y el análisis de un dataset de libre acceso, aplicando los conocimientos adquiridos.
5. **Adquirir** experiencia práctica a partir de ejercicios y ejemplos fundamentados en los materiales en PDF que has proporcionado.

---

2. Estructura y cronograma propuestos (7 semanas)

1. **Semana 1**: Configuración del entorno y fundamentos de Python
2. **Semana 2**: Estructuras de datos nativas en Python y control de flujo
3. **Semana 3**: Introducción a NumPy
4. **Semana 4**: Introducción a pandas: carga de datos y primeros pasos
5. **Semana 5**: Limpieza y manipulación de datos con pandas
6. **Semana 6**: Visualización y análisis exploratorio de datos (EDA)
7. **Semana 7**: Proyecto integrador y cierre

---

3. Detalle de contenidos, guías de estudio y actividades prácticas por semana

A continuación, se describen los contenidos principales de cada semana, las guías de estudio (incluyendo las referencias PDF que proporcionaste) y sugerencias de prácticas. Se busca una combinación teórico-práctica que permita a los participantes afianzar conocimientos a partir de la aplicación inmediata de los conceptos.

Semana 1: Configuración del entorno y fundamentos de Python

- **Contenidos**
    - Presentación del curso: alcance, metodología y evaluación.
    - Instalación y configuración: Anaconda / Python oficial.
    - Uso básico de Jupyter Notebook / Google Colab / VS Code.

- Repaso de la Terminal (breve introducción, si se requiere).
- Primeros pasos: ejecución de sentencias simples (p. ej., print()).
- Revisión del material *Programa Académico - Programación con Python para el análisis de datos* (PDF adjunto) para ubicar los temas principales.

- **Guía de estudio**
  - **Lecturas recomendadas**:
    1. *Programa Académico - Programación con Python para el análisis de datos* (secciones iniciales).
    2. *chapter1.pdf* (introducción a Python: nociones y ejemplos de configuración).
  - **Actividades prácticas**:
    1. **Verificación de instalación**: Abrir un Notebook y hacer un "Hola Mundo", exportarlo a PDF.
    2. **Explorar Kaggle**: Crear cuenta y examinar datasets simples (p. ej., *Iris Dataset*).

Semana 2: Estructuras de datos nativas en Python y control de flujo

- **Contenidos**
  - Tipos de datos: numéricos, booleanos, cadenas de texto (strings).
  - Variables y operadores (aritméticos, relacionales, lógicos).
  - Estructuras de control: if-else, elif, bucles for y while.
  - Estructuras de datos nativas: listas, tuplas, conjuntos y diccionarios.
  - Diferencias básicas entre cada estructura y sus métodos más usados.
- **Guía de estudio**
  - **Lecturas recomendadas**:
  - **Actividades prácticas**:
    1. **Gestor de contactos**: Usar un diccionario para almacenar datos de contactos y un pequeño menú en consola para agregar, buscar y editar.
    2. **Juego de adivinanza**: Implementar un bucle while que genere un número aleatorio y el usuario intente adivinarlo. Se mostrarán pistas.

3. **Mini-desafíos**: Ejercicios de manipulación de listas (inversión, corte, concatenación), y de diccionarios (inserción, eliminación).

Semana 3: Introducción a NumPy

- **Contenidos**
  - Importar la librería NumPy.
  - Creación y manipulación de arreglos (arrays): np.array(), np.zeros(), np.ones(), np.random().
  - Indexación y slicing en NumPy.
  - Operaciones vectorizadas, estadísticas básicas (suma, media, desvío estándar).
  - Comparación entre listas de Python y arrays de NumPy en términos de eficiencia.
- **Guía de estudio**
  - **Lecturas recomendadas**:
  - **Actividades prácticas**:
    1. **Ejercicios con arrays**: Crear arrays 1D y 2D, practicar slicing, operaciones matemáticas y reducción (sumas por filas o columnas).
    2. **Simulaciones**: Generar datos aleatorios y calcular media, desviación estándar y percentiles.
    3. **Comparación**: Implementar en listas puras una operación y luego en NumPy, midiendo tiempos (básico) para notar la diferencia.

Semana 4: Introducción a pandas: carga de datos y primeros pasos

- **Contenidos**
  - Presentación de pandas: Series y DataFrame.
  - Lectura y escritura de archivos CSV (y brevemente Excel).
  - Inspección de datos: .head(), .tail(), .info(), .describe().
  - Selección y filtrado de datos: .loc[], .iloc[], y condiciones booleanas.
- **Guía de estudio**
  - **Lecturas recomendadas**:

- **Actividades prácticas**:
    1. **Carga de un dataset**: Descargar un CSV (por ejemplo, *Titanic*) y explorarlo con las funciones mencionadas.
    2. **Filtrado**: Seleccionar filas basándose en condiciones (p. ej., pasajeros mayores de 30 años o películas de cierto género).
    3. **Exportación**: Guardar un subconjunto o DataFrame filtrado a CSV.

Semana 5: Limpieza y manipulación de datos con pandas

- **Contenidos**
    - Tratamiento de datos faltantes: métodos para detectar (isnull()) y rellenar (fillna()) o eliminar (dropna()).
    - Conversión de tipos (texto a numérico, fechas, categórico).
    - Agrupaciones y agregaciones: groupby(), sumas, medias, conteos.
    - Uniones de DataFrames: merge(), concat().
    - Creación de columnas derivadas y uso de apply() / funciones lambda.
- **Guía de estudio**
    - **Lecturas recomendadas**:
    - **Actividades prácticas**:
        1. **Limpieza en Titanic**: Examinar valores nulos en “Age”, decidir estrategia de imputación; mostrar antes y después.
        2. **Columnas derivadas**: Calcular una columna nueva, por ejemplo, “Edad en décadas”.
        3. **Merge y concat**: Combinar datos de dos fuentes (ej. datos de películas y datos de ratings).

Semana 6: Visualización y análisis exploratorio de datos (EDA)

- **Contenidos**
    - Fundamentos de Matplotlib: plt.plot(), plt.scatter(), plt.bar(), plt.hist().
    - Personalización de gráficos (etiquetas, leyendas, ejes, guardado de figuras).
    - Estadística descriptiva básica (media, mediana, moda, desviación estándar, correlaciones simples).
    - Boxplots y detección de outliers de manera introductoria.

- Aplicación de las técnicas de EDA para formular y responder preguntas sobre un dataset.

- **Guía de estudio**
  - **Lecturas recomendadas**:
  - **Actividades prácticas**:
    1. **Gráficos básicos**: Para un dataset (Titanic o Iris), graficar histogramas de sus variables numéricas y scatter plots entre dos columnas.
    2. **EDA con boxplots**: Detección de valores atípicos en una o varias variables, análisis de correlaciones simples (df.corr()).
    3. **Informe exploratorio**: Generar un Notebook con tablas de resumen y visualizaciones; exportar a PDF.

Semana 7: Proyecto integrador y cierre

- **Contenidos**
  - Selección de un dataset (propio o de Kaggle/otro repositorio).
  - Aplicación de todo el pipeline:
    1. Carga y limpieza de datos.
    2. Exploración y visualización inicial (EDA).
    3. Análisis y conclusiones finales.
  - Presentación final de resultados y retroalimentación.

- **Actividades prácticas**

1. **Proyecto integrador**:
   - Cada participante o equipo elige un dataset.
   - Implementa limpieza, transformación y generación de visualizaciones significativas.
   - Elabora un reporte final en Jupyter Notebook con explicación y resultados, exportado a PDF.

2. **Exposición**: Breve presentación oral de cada proyecto, subrayando las principales conclusiones y retos encontrados.

3. **Evaluación y Feedback**: Se sugiere emplear una rúbrica que considere criterios como la coherencia del análisis, la calidad del código y la claridad de la comunicación.

---

4. Referencias y materiales de apoyo

1. **Materiales PDF compartidos (fuente principal para el diseño del curso)**:
2. **Documentación oficial**:
    - Python: https://docs.python.org/3/
    - NumPy: https://numpy.org/doc/stable/
    - pandas: https://pandas.pydata.org/docs/
    - Matplotlib: https://matplotlib.org/stable/contents.html
3. **Libros y guías en línea**:
    - Sweigart, A. (2019). *Automate the Boring Stuff with Python*.
    - McKinney, W. (2017). *Python for Data Analysis*.

---

5. Método de evaluación y consideraciones finales

1. **Evaluaciones parciales**:
    - Tareas al final de cada semana, enfocadas en la práctica directa (miniproyectos de manipulación o visualización de datos).
2. **Proyecto Final**:
    - El informe en Notebook + PDF (incluye aspectos de limpieza, análisis y visualizaciones).
    - La presentación oral (si las dinámicas de la asignatura lo permiten).
3. **Organización**:
    - Se sugiere el uso de un repositorio compartido (GitHub) para que los estudiantes practiquen control de versiones.
    - Mantener carpetas estructuradas (datos, scripts, notebooks, etc.).
4. **Opinión y sugerencia de ChatGPT**:
    - Reforzar la práctica continua. Si el tiempo lo permite, se pueden introducir nociones básicas de “Storytelling con datos” para una mejor comunicación de resultados.
    - Motivar a quienes tengan más tiempo e interés a explorar librerías complementarias como Plotly para gráficos interactivos o a cursar un siguiente módulo enfocado en Machine Learning.